

API DESIGN

¿Quién es Wesley beary?

Wesley Beary es co-fundador de Anchor.dev (una herramienta para desarrolladores que sirve para **gestionar certificados y usar HTTPS de forma fácil**, especialmente en desarrollo) y ha trabajado como ingeniero y desarrollador en empresas como Salesforce, Heroku y Engine Yard. Especialista en diseño de APIs. Ha creado guías de buenas prácticas (como la de Heroku). Tiene experiencia real construyendo APIs usadas por muchos desarrolladores.

¿Qué es una API?

Una API, *Application Programming Interface*, es la interfaz que exponemos a usuarios externos para que interactúen con nuestro sistema. Puede ser una web API sobre HTTP, una librería, o una interfaz de línea de comandos.

Wesley tiene su propio acrónimo alternativo para API: **Assumptions Probably Incorrect**, es decir, "suposiciones probablemente incorrectas". ¿Por qué? Porque según él, y según su experiencia real, ningún plan de interfaz sobrevive al contacto con la realidad. Siempre que diseñas una API de cero, hay cosas que se te olvidan, casos extremos que no consideras, y el resultado final nunca es tan bueno como pensabas en el papel.

EL PELIGRO DE LAS SUPOSICIONES

Si las suposiciones siempre son incorrectas, la pregunta es: ¿qué hacemos con eso?

La respuesta de Wesley Beary es bastante clara: en lugar de planificar demasiado, hay que iterar rápido.

El enfoque tradicional es diseñar todo desde el principio, implementarlo completo y después descubrir errores de diseño, cuando ya es demasiado tarde y el coste de cambiarlos es muy alto.

En cambio, Wesley propone empezar con una especificación ligera (documento que define cómo se comporta una API antes de implementarla), crear un prototipo rápido y usarlo en situaciones reales.

Y a partir de ahí, ir iterando con el feedback que obtienes.

Esto tiene varias ventajas: detectas errores mucho antes, cuando todavía es barato cambiarlos.

Te aseguras de que la API realmente funciona para los usuarios, no solo en teoría.

Reducas el riesgo de tener que rehacer grandes partes del sistema más adelante.

¿Cómo obtener feedback real?

Vale, por lo tanto necesitamos iterar con feedback. Pero ¿cómo obtenemos feedback real de nuestros usuarios?

La idea es sencilla:

Sientas a un usuario junto con tu API, la documentación de la API y una tarea a resolver. Le dices que piense en voz alta y tu te callas no le das ninguna ayuda, te limitas a observar como se desenvuelve en la resolución de la tarea. Idealmente grabas la sesión para poder revisarla después. ¿Por qué es tan importante no intervenir? Porque si ayudas al usuario cuando se atasca, estás "guiando al testigo". Estás invalidando el test. El problema real es que cuando tu API esté en producción, tú no vas a estar en la sala para rescatar a nadie. El usuario tiene que conseguir resolver la tarea solo con tu API. Así podemos observar en que puntos tiene dificultades el usuario y si haciendo la prueba con varios usuarios distinto todos se atascan en el mismo punto, es una señal clarísima de que necesitamos mejorarlo.

Además, Wesley señala algo muy importante: cuando observas cómo la gente habla sobre lo que intenta hacer, muchas veces descubres que usan una terminología completamente diferente a la tuya. Y eso te dice que quizás tu documentación está usando los términos equivocados para tu audiencia.

Más allá de CRUD

Muchas APIs se comportan básicamente como un CRUD y ya. Solamente tiene las operaciones de creación lectura actualización y borrado. EL problema de esto viene cuando intentamos que estas cuatro operaciones cubran absolutamente todo. Acabamos teniendo métodos muy sobrecargados y complejos. Para poder verificar si estamos diseñando bien la API debemos preguntarnos que hace realmente esta operación, si la respuesta es Depende, estamos haciendo un mal diseño de la API, Si el comportamiento depende de qué parámetros mandas, estás haciendo muy difícil que alguien razone sobre tu interfaz.

¿Cómo soluciona esto Wesley? Con el **patrón de acciones**. Básicamente es crear acciones específicas que complementen al CRUD y separar las acciones en su propio namespace dentro de la URL. Es decir en vez de tener un endpoint que sea `servers/id` con estado = reiniciando, hacer un endpoint que sea `/servers/{id}/actions/restart`.

Las ventajas del patrón de acciones son que hace que la API sea mucho más clara y explícita, porque cada acción tiene un nombre que describe exactamente lo que hace. evita sobrecargar el CRUD. mejora la mantenibilidad, porque cada acción está separada y es más fácil de modificar o extender. facilita el uso por parte de otros desarrolladores, ya que es más intuitivo y fácil de entender

PRINCIPIO de mínima sorpresa

El siguiente concepto es el principio de mínima sorpresa, La idea es que si un usuario aprende cómo funciona una parte de tu API, debería poder adivinar cómo funciona el resto, sin sorpresas.

Veamos tres ejemplos concretos. Primero, el naming inconsistente: si en un recurso usas DELETE para eliminar y en otro recurso usas REMOVE, el usuario inevitablemente se pregunta: ¿hay alguna diferencia semántica entre estos dos? ¿Remove borra el objeto o solo desvincula la asociación? Esa duda genera desconfianza en toda la API.

Comandos como reset, revert y restore hacen cosas que se parecen pero que son sutilmente distintas, y es muy difícil recordar cuál hace qué.

los mensajes de error sin contexto. Un "Something went wrong, please try again" es básicamente inútil. Un mensaje de error debe especificar el error y probablemente también cómo arreglarlo. La diferencia en experiencia de usuario es enorme.

Con el principio de mínima sorpresa surge la duda de si es bueno innovar ya que si hacemos una API nueva e innovadora se rompe este principio. La respuesta es que hacer una API innovadora es muy probable que salga mal. Si queremos hacer una API diferente tiene que haber un equilibrio entre lo innovador y lo que el usuario ya conoce, para que el cliente pueda entender la API y se cumpla este principio.

Versionado y documentación

Una vez que una versión de tu API está en producción y tiene usuarios, tienes que mantenerla. No puedes ir a cambiar el código de tus usuarios. Para ello va a haber que ir actualizándola y sacando versiones, pero sacar versiones se debe evitar en medida de lo posible. Si sacas una v2, tienes que seguir manteniendo la v1 durante meses o años. El coste operativo se multiplica. Por eso su recomendación es centrarse en hacer solo cambios no rompedores que sean compatibles con la versión actual. Si sacamos alguna versión nueva es importante darle tiempo al usuario para migrar a la nueva versión.

En cuanto a la documentación, destaca un problema muy común: muchas veces se escribe de forma manual y acaba quedándose desactualizada.

Para evitar esto, recomienda usar OpenAPI,

es un estándar que permite describir una API de forma estructurada, de manera que se pueda documentar, probar y hasta simular automáticamente.

De esta forma, la documentación se genera automáticamente a partir de la especificación, y además se pueden validar tests para asegurar que lo implementado coincide con lo documentado. También se pueden usar mockServer que son como servidores falsos que simulan una API real

Así se evita que haya diferencias entre lo que la API dice que hace y lo que realmente hace.

La documentación no debería escribirse aparte, sino generarse directamente desde la propia definición de la API.

Errores al diseñar APIs

Mencionar los errores de que hemos ido comentado al lo largo de la presentación

Hay que tener en cuenta que los usuarios no conocen como nosotros los desarrolladores el dominio para el cual esta diseñada la API, incluso si la exponemos de una manera que consideramos sencilla , para los usuarios puede no serlo. Estamos suponiendo que tienen una experiencia que tal vez no tengan. No hay que pensar solo en que la API pueda técnicamente hacer de todo sino también en lo que será fácil de entender y descubrir si nunca se ha utilizado antes. Algunas partes de la API deben ir dirigidas solo para usuarios nuevos sin experiencia, para proporcionarles apoyo.

Intentar crear algo que sea suficientemente bueno para todos no suele ser una buena opción porque acaba siendo tan flexible que resulta complejo y termina no siendo bueno para nadie. A la hora de diseñar una API hay que profundizar también en los detalles , probarlo y usarlo en situaciones difíciles como si fueras un usuario, así se revelan cosas que no habíamos pensado.